

道案内

台湾には高山が密集しており、標高 3000 メートルを超える山が 250 以上ある． A-Ming はロボットによるジップライン配送ネットワークを台湾の中央山脈に構築し、遠方の村同士で荷物を運ぶ計画を立てている．

A-Ming の計画では、各ネットワークは N 個のノードからなり、それらには 0 から $N - 1$ までの番号が付けられている．これらのノードのうち、ちょうど $K = 6$ 個が**発電所**であり、残りの $N - K$ 個のノードはすべて**村ノード**である．ノード同士は $N - 1$ 本の双方向のケーブルで接続されており、ケーブルには 0 から $N - 2$ までの番号が付けられている．各 i ($0 \leq i \leq N - 2$) について、ケーブル i はノード $U[i]$ と $V[i]$ を結ぶ．各ケーブルは異なる 2 つのノードを結び、任意の 2 つのノードの間には高々 1 本のケーブルしかない．いくつかのケーブルを辿ることで任意の 2 つのノードの間を移動できることが保証されている．

ノード a と b の**距離**を $d(a, b)$ と書き、次のように定義する．

- $a = b$ のとき、 $d(a, b) = 0$ ．
- そうでないとき、 $d(a, b)$ はノード a から b へ移動するのに必要なケーブルの本数の最小値である．

各ノードが、ちょうど 1 本あるいは δ 本以上のケーブルに接続されているとき、ネットワークの**分岐度**が δ 以上であるという． A-Ming は、ネットワークの分岐度が B 以上であるような正の整数 B を知っている．

A-Ming は番号 $0, 1, \dots, 99$ からなる 100 種類のケーブルを用意した．ネットワークの各ケーブルは、これらのいずれかの種類である．

ロボットはジップラインのケーブルに沿って移動し、配送業務を行う． A-Ming は、ロボットが発電所に戻って充電できるように、道案内プログラムを組み込みたいと考えている．しかし、ロボットの搭載メモリは極めて小さい．そのため道案内プログラムの設計では、ロボットは発電所の個数 $K = 6$ 、分岐度が B 以上であることを表す正の整数 B 、およびロボットの現在地に接続されているケーブルの種類の情報だけに基づいて移動する．

ロボットが 2 本以上のケーブルに接続された**村ノード** u で道案内をしようとするとき、ロボットはまず u に接続されているケーブルを任意の順で走査する．その順番に従って、ケーブルの種類の列がプログラムに渡される．プログラムは各ケーブルについて、そのケーブルに沿って進んだ時にロボットと発電所との距離が小さくなるような発電所の個数を数えなければならない．

具体的には、 u とケーブルで直接結ばれているノードを $v_1, \dots, v_k$ ($k \geq 2$) とし、ノード u と v_i を結ぶケーブルの種類を c_i ($1 \leq i \leq k$) とする。プログラムには K, B 、および列 $c_1, c_2, \dots, c_k$ が渡される。各 i ($1 \leq i \leq k$) に対して、プログラムは、 $d(v_i, p) < d(u, p)$ を満たす発電所 p の個数を求めなければならない。

あなたの仕事は、ケーブルの種類の割り当て方と道案内プログラムの設計方針を考えることである。解法の得点は、用いるケーブルの種類の個数（詳細は小課題と採点の節を参照）に依存する。

実装の詳細

ケーブルの種類を割り当てる関数と、ロボットのプログラムの関数の2つを実装する必要がある。

ケーブルの種類を割り当てるために実装すべき関数は次のとおりである。

```
std::vector<int> construct_network(int N, int K, int B,
                                   std::vector<int> U, std::vector<int> V,
                                   std::vector<int> P)
```

- N ：ノードの個数。
- K ：発電所の個数。
- B ：ネットワークの分岐度が B 以上であることを表す。
- U, V ：長さ $N - 1$ の配列で、ケーブルによる接続を表す。
- P ：長さ $K = 6$ の配列で、発電所であるノードを表す。
- この関数は各テストケースにつき **高々 10 000 回** 呼び出される。

この関数は長さ $N - 1$ の配列 T を返さなければならない。

- 種類 $T[i]$ のケーブルがノード $U[i]$ と $V[i]$ を結ぶことを意味する。
- 各要素 $T[i]$ は $0 \leq T[i] < 100$ を満たさなければならない。

ロボット用のプログラムとして実装すべき関数は次のとおりである。

```
std::vector<int> navigate(int K, int B, std::vector<int> C)
```

- K ：発電所の個数。
- B ：ネットワークの分岐度が B 以上であることを表す。
- C ：ある **2 本以上のケーブルに接続された村ノード** について、それに接続されているすべてのケーブルの種類を任意の順で並べた列。
- C の長さは B 以上であることが保証されている。
- この関数は各テストケースにつき **高々 100 000 回** 呼び出される。
- 関数 `navigate` は元のネットワークの構造に依存してはならない。特に、採点システムは同一テストケース内の異なる構築済みネットワークにわたる `navigate` の呼び出しを並べ替えることが

ある.

この関数は配列 D を返さなければならない.

- 配列 D の長さは配列 C の長さと同じでなければならない.
- $D[i]$ は, $C[i]$ に対応するケーブルに沿って進んだ時に, ロボットと発電所との距離が小さくなるような発電所の個数でなければならない.

採点システムでは, `construct_network` と `navigate` が 2 つの別々のプロセスで呼び出されることに注意せよ.

制約

- $K = 6$.
- $K < N \leq 100\,000$.
- 各テストケースにおいて, `construct_network` のすべての呼び出しにわたる N の総和は $100\,000$ を超えない.
- 各 $0 \leq i \leq N - 2$ について, $0 \leq U[i] < V[i] < N$.
- いくつかのケーブルを辿ることで任意のノードから任意のノードへ移動できる.
- $0 \leq P[0] < P[1] < \dots < P[K - 1] < N$.
- 各テストケースにおいて, `navigate` のすべての呼び出しにわたる配列 C の長さの総和は $200\,000$ を超えない.
- 各配列 C は, ある 2 本以上のケーブルに接続された村ノードについて, それに接続されているすべてのケーブルの種類を任意の順で並べたものである.

小課題と採点

小課題	得点	追加の制約
1	6	$B = 2, N = 7$.
2	14	$B = 2$, 各 i ($0 \leq i < N - 1$) について $U[i] = i, V[i] = i + 1$ が成り立つ.
3	20	$B = 5$.
4	20	$B = 4$.
5	40	$B = 2$.

いずれかのテストケースにおいて, 次のいずれかが成り立つならば, そのテストケースにおける得点は 0 点 (CMS では `Output isn't correct` として報告される) となる.

- `construct_network` のいずれかの呼び出しの戻り値が無効である.
- `navigate` のいずれかの呼び出しの戻り値が正しくない.

そうでない場合， S を，`construct_network` の各呼び出しから返されたすべての配列 T に現れる数のすべてより大きい整数のうち最小のものとする．言い換えると， S は，構築されたすべてのネットワークが種類 $0, 1, \dots, S - 1$ のケーブルだけを用いるような最小の整数である．

各小課題の得点は S に応じて次のように決まる．

条件	小課題 1	小課題 2	小課題 3 および 4	小課題 5
$100 < S$	0	0	0	0
$16 \leq S \leq 100$	2	2	$12 - \log_2 S$	$24 - 2 \log_2 S$
$10 \leq S \leq 15$	2	4	$16 - 0.5S$	$32 - S$
$7 \leq S \leq 9$	2	6	$21 - S$	$42 - 2S$
$S = 6$	2	10	15	30
$S = 5$	6	14	17.5	40
$S \leq 4$	6	14	20	40

特に，小課題 1，2，5 では $S \leq 5$ なら満点となり，小課題 3 および 4 では $S \leq 4$ なら満点となる．

例

$N = 9, K = 6, B = 2$ の状況を考える．

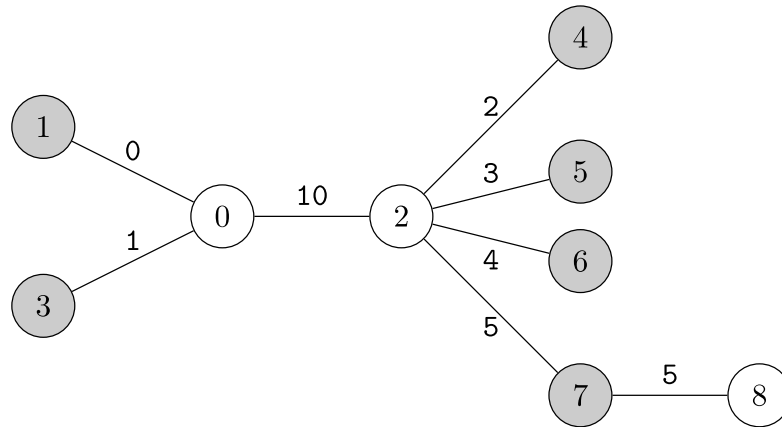
ネットワークの設計は $U = [0, 0, 0, 2, 2, 2, 2, 7], V = [1, 2, 3, 4, 5, 6, 7, 8]$ で与えられる．発電所は $P = [1, 3, 4, 5, 6, 7]$ である．最初のプロセスにおける次の呼び出しを考える．

```
construct_network(9, 6, 2, [0, 0, 0, 2, 2, 2, 2, 7],
                    [1, 2, 3, 4, 5, 6, 7, 8],
                    [1, 3, 4, 5, 6, 7])
```

A-Ming が次の配列を返してネットワークを構築したとする．

```
[0, 10, 1, 2, 3, 4, 5, 5]
```

構築されたネットワークは次の図のとおりである．



続いて、ロボットがノード 0 で道案内をする必要があるとする． ノード 0 はノード 1, 2 および 3 と接続されている． ロボットがケーブルの種類をこの順で読み取ったとすると、次の呼び出しが行われる．

```
navigate(6, 2, [0, 10, 1])
```

ネットワークの構造に基づく次のとおりである．

- 最初の発電所であるノード 1 は、ノード 1 に進んだ時のみ距離が短くなる．
具体的には、 $0 = d(1, 1) < 1 = d(0, 1) < 2 = d(2, 1) = d(3, 1)$ ．
- 次の発電所であるノード 3 は、ノード 3 に進んだ時のみ距離が短くなる．
具体的には、 $0 = d(3, 3) < 1 = d(0, 3) < 2 = d(1, 3) = d(2, 3)$ ．
- その他の発電所であるノード 4, 5, 6, 7 は、ノード 2 に進んだ時のみ距離が短くなる．
具体的には、 $u = 4, 5, 6, 7$ のとき、 $1 = d(2, u) < 2 = d(0, u) < 3 = d(1, u) = d(3, u)$ ．

ケーブル (0,1), (0,2), (0,3) に沿って進んだ時に距離が小さくなる発電所の個数は、それぞれ 1, 4, 1 である．したがって、この関数は次を返さなければならない．

```
[1, 4, 1]
```

`navigate` は C の別の並び替えに対しても呼び出されうることに注意せよ．例えば、ノード 0 に対して `navigate(6, 2, [1, 0, 10])` もあり得る呼び出しである．この場合、関数はそれに応じて `[1, 1, 4]` を返さなければならない．

次に、ロボットがノード 2 で道案内をする必要があるとする． 次の呼び出しが行われる．

```
navigate(6, 2, [10, 2, 3, 4, 5])
```

関数は次を返さなければならない．

```
[2, 1, 1, 1, 1]
```

このネットワークでは次の理由から、ほかのノードで `navigate` が呼び出されることはない。なぜならば、

- ノード 1, 3, 4, 5, 6, 7 はすべて発電所であり、
- ノード 8 は 1 本のケーブルにしか接続されていない。

からである。

このテストケースでは、用いられたケーブルの種類は 0, 1, 2, 3, 4, 5, 10 である。したがって、得点の決定には $S = 11$ が用いられる。

この問題の添付パッケージには、この例に加えて、採点プログラムのサンプル用のほかに 2 組のサンプル入出力が含まれている。

採点プログラムのサンプル

採点プログラムのサンプルは同一プロセス内で `construct_network` と `navigate` の両方を呼び出す。さらに、採点プログラムのサンプルが `navigate` を呼び出すとき、ケーブルの種類は入力において対応するケーブルが現れる順に列挙される。これらの挙動は本番の採点プログラムとは異なる。

入力形式：

```
T
(scenario 0)
(scenario 1)
...
(scenario T-1)
```

各シナリオは次の形式である。

```
N
B
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
K
P[0] P[1] ... P[K-1]
Q
X[0] X[1] ... X[Q-1]
```

各シナリオにおいて、採点プログラムのサンプルは `navigate` を Q 回呼び出し、 i 回目の呼び出しではノード $X[i - 1]$ の情報を渡す。採点プログラムのサンプルは K に 6 以外の整数を設定することもできることに注意せよ。

出力形式：

`construct_network` または `navigate` の戻り配列のいずれかが無効であれば、採点プログラムのサンプルは中止し、理由を出力する。そうでない場合、採点プログラムのサンプルは `navigate` の呼び出しから返された各配列 D を 1 行に出力する。

すべてのシナリオの処理が終わったあと、採点プログラムのサンプルは標準エラー出力に次の 1 行を出力する。

```
S = S'
```

ここで、 S' はそのテストケースにおいて `construct_network` の各呼び出しから返されたすべての配列 T に現れる数のすべてより大きい整数のうち最小のものである。